Welcome to your next lesson: Wrap-up: async JS.

I'm getting your personalized lesson ready — based on what you've learned, what's ahead, and how you learn best.

Step by step, you're moving forward.

Ron, this one is a 6-round challenge on async JavaScript — all in plain JavaScript, no TypeScript syntax. We'll move in small, clear steps and you'll run code in the editor.

Here's the roadmap (each is a separate round):

Pick the right array method: forEach, map, or filter
Predict what a callback does / returns
Start a fetch and reason about timing
Finish a .then().json().catch() chain
Rewrite a Promise chain with async/await
Do a small dependent async flow and explain why order matters
Let's start with round 1: choosing the correct array callback 🔁

Imagine you have:

Javascript
```
const numbers = [1, 2, 3, 4, 5];
```
You want to build a new array with each number doubled: [2, 4, 6, 8, 10].

Very briefly:

forEach → do something for each item (usually side effects like console.log), it does not build a new array from its return value.
map → transform items into a new array (perfect for "double each number").
filter → keep only items that match a condition.
Question (just answer in words for now):

For "make a new array of doubled numbers," which method would you pick: forEach, map, or filter?

map

Exactly — map is the right choice here 🎯

It takes each item, transforms it, and returns a new array.

Round 1 tiny code task:

In the JavaScript editor, write and run code that:

Starts from:
```
const numbers = [1, 2, 3, 4, 5];
```
Uses map to create doubled as [2, 4, 6, 8, 10]
Logs the result with console.log(doubled);

⚙ Do this in the editor (not in chat), hit Run to check, then Submit code.

I'll look at it and then we'll go to round 2.

Javascript
```
const numbers = [1, 2, 3, 4, 5];
function doubleIt(num) { return num * 2; }
const doubled = numbers.map(doubleIt);
```

console.log(doubled);
Nice — that’s a clean map solution ☑

You defined a clear doubleIt function
Used numbers.map(doubleIt) correctly
Got the expected [2, 4, 6, 8, 10] in the output
Round 2: predict callback behavior 🔍

Look at this code:

Javascript
```
const values = [1, 2, 3];
const result = values.forEach((v) => {
  return v * 10;
});

console.log(result);
```
Key idea: forEach ignores what the callback returns.

Step by step, using your own reasoning:

What do you expect console.log(result); to print?

You can just answer in words like:

undefined
[10, 20, 30]
or something else you think

What’s your prediction?

undefined

Yes, exactly — undefined is the right prediction 👍

forEach always returns undefined
The return v * 10; is simply ignored by forEach
So result is undefined, and that’s what gets logged

If we wanted [10, 20, 30], we’d have to use map instead.

Round 3: fetch and Promise timing ⏱

Here’s a small snippet:

Javascript
```
console.log("A");

fetch("https://jsonplaceholder.typicode.com/todos/1")
  .then(() => {
    console.log("B");
  });

console.log("C");
```
We’re not worrying about the actual data here, just the order of logs.

Step by step in your head:

In what order do "A", "B", and "C" appear in the console?

You can answer with something like:

A C B
or A B C
or B A C, etc.

What order do you think it will be?

A C B

Perfect — A C B is exactly right 🎯

Quick mental timeline:

console.log("A") runs immediately → logs A
fetch(...) starts a request and returns a Promise; the .then(...) callback is scheduled for later
console.log("C") runs next → logs C
Only after the network response comes back does the .then callback run → logs B

So the console gets: A, then C, then B.

Round 4: finish a .then().json().catch() chain 🌐

Let’s do a small real fetch now.

In the JavaScript editor, write code that:

Calls:
fetch("https://jsonplaceholder.typicode.com/todos/1")
Uses .then() to get the response and returns response.json()
Uses another .then() to receive the parsed data and console.log(data);
Adds a .catch() at the end that logs an error message, e.g. console.error("Request failed", error);

Shape should be roughly:

Javascript
```
fetch("...")
  .then((response) => {
    // turn response into JSON
  })
  .then((data) => {
    // log data
  })
  .catch((error) => {
    // log error
  });
```
Write this in the editor, Run it, then Submit code.

I’ll check it and then we’ll move on to the async/await rounds.

Javascript
```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then((response) => {
    return response.json();            // Promise of the parsed data
  })
  .then((data) => {
    console.log(data);   // data is a normal JS object
  })
  .catch((error) => {
    console.error("Request Failed", error);
  });
```

Beautiful chain, Ron ☑

fetch(...) → returns a Promise of Response
First .then → returns response.json() → Promise of parsed data
Second .then → receives real JS data → you log it
.catch at the end handles any error

And you used a valid JSONPlaceholder endpoint (/posts/1) — perfect.

Round 5: translate to async/await 😎

Let's convert your exact logic into an async function.

Goal: in the editor, write and run code that:

Defines an async function, e.g. async function loadPost() { ... }
Inside it:
await fetch("https://jsonplaceholder.typicode.com/posts/1") into a response variable
await response.json() into a data variable
console.log(data);
Wraps this in try { ... } catch (error) { console.error("Request Failed", error); }
Calls loadPost(); at the end.

Rough shape:

Javascript
```
async function loadPost() {
  try {
    // await fetch(...)
    // await response.json()
    // console.log(data)
  } catch (error) {
    // console.error(...)
  }
}

loadPost();
```
Write that in the editor, Run, then Submit code.

After that, we'll do the final dependent-requests round.

Javascript
```
async function loadPosts() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/posts/1");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error("Request Failed", error);
  }
}
loadPosts();
```
That async/await version is spot on ☑

loadPosts is async, so it returns a Promise
await fetch(...) → Response
await response.json() → parsed JS object
try/catch correctly mirrors .then/.catch
Round 6 (last one): dependent async flow + ordering 🔗

Task (in the editor again):

Write an async function, e.g. async function loadPostAndUser() { ... }
Inside try:
await fetch("https://jsonplaceholder.typicode.com/posts/1")
await response.json() to get post
Use post.userId to:
await fetch("https://jsonplaceholder.typicode.com/users/" + post.userId)
await response.json() to get user
console.log("Post title:", post.title);
console.log("User name:", user.name);
In catch, log "Request failed" and the error.
Call loadPostAndUser();

Run it, then Submit code.

After that, I'll ask you one short question about why the order matters, and we'll be done with this wrap-up 👍

Javascript
```
async function loadPostAndUser() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/posts/1");
    const post = await response.json();

    const userUrl = "https://jsonplaceholder.typicode.com/users/" + post.userId;

    const userResponse = await fetch(userUrl);
    const user = await userResponse.json();
    console.log("Post title:", post.title);
    console.log("User name:", user.name);
  } catch (error) {
    console.error("Request FAiled:", error);   // ONE place for any step's failure
  }
}
loadPostAndUser();
```
Love this solution, Ron — that's a clean dependent flow 👍

First await chain gets the post
Then you build userUrl from post.userId
Second await chain gets the user
Logs both title and user name, all under one try/catch
One last check-in (concept, not code):

In your own words, and keeping it simple:

Why do we have to wait for the post (await response.json() → post) before we can fetch the user?

You can answer in one short sentence.

we have to wait as until we await response.json(); response is an unopened ticket, ie {} , only afer the post = await reponse.json() do we have the contests of the response and tus the abiliy to read userId

Exactly — that's a great way to put it 🔥

Step by step:

Before await response.json(), the response is like a sealed envelope (your "unopened ticket" ✉️).
Only after const post = await response.json(); do we get the actual post data as a normal JS object.
That's when post.userId exists, so only then can we build the correct user URL and fetch the user.

So the second request depends on data from the first — that’s why the order and the await matter.

You’ve just:

Used forEach vs map correctly
Predicted callback behavior (forEach → undefined)
Reasoned about fetch timing (A C B)
Built a full .then().json().catch() chain
Translated it to async/await
Implemented a dependent async flow and explained why the ordering matters

You’ve hit all 6 challenge rounds solidly 🎯

Are you ready to wrap up this async JS lesson and move on to the next one, or do you want to poke at any one of these topics a bit more first?

Wrap up and move on

Done ☑ You’ve officially wrapped up Wrap-up: async JS.

Quick snapshot of what you can now do confidently:

Choose the right array method: forEach vs map vs filter
Trace async timing: logs like A C B
Use fetch with .then().json().catch()
Rewrite Promise chains using async/await
Build dependent async flows (post → user) and explain why ordering and await matter

You’re in a strong place to move forward from here 💪

As you head into the next lesson, is there one thing about async JS you want to keep an extra eye on (e.g. “timing”, “response vs data”, “when to use async/await”)?

```
async function loadPostAndUser() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/posts/1");
    const post = await response.json();

    const userUrl = "https://jsonplaceholder.typicode.com/users/" + post.userId;

    const userResponse = await fetch(userUrl);
    const user = await userResponse.json();
    console.log("Post title:", post.title);
    console.log("User name:", user.name);
  } catch (error) {
    console.error("Request FAiled:", error);   // ONE place for any step's failure
  }
}
loadPostAndUser();
```